

# GNN-Based Recommendation System

Complete Study & Reference Guide

LightGCN · PyTorch Geometric · MovieLens 100K

|                          |                 |
|--------------------------|-----------------|
| Recall@20                | 0.1348          |
| NDCG@20                  | 0.0555          |
| Anomaly Users<br>Flagged | 48 / 943 (5.1%) |

## Part 1 — Conceptual & Theory Questions

---

### Q1. What is a Graph Neural Network and why is it better than a normal neural network for recommendations?

A normal neural network takes a **flat table** as input — rows and columns. It cannot capture **relationships** between entities. A GNN takes a **graph as input** — nodes and edges — and uses **message passing** where each node learns from its neighbours.

#### Why this matters for recommendations

In recommendations, a user node learns from movies it watched, and a movie node learns from users who watched it.

This captures collaborative patterns that a flat model completely misses.

After K layers, each user has 'seen' information from K hops away — friends of friends of friends.

### Q2. What is message passing in a GNN?

Message passing is the core operation of any GNN. In each layer:

1. Every node collects feature vectors from its neighbours
2. It aggregates them (mean, sum, or max)
3. It updates its own embedding using that aggregated information

```
Layer 1: User sees direct movies it watched
Layer 2: User sees movies watched by similar users
Layer 3: User sees even more distant but related patterns
```

### Q3. What is a bipartite graph and why do we use it in recommendation systems?

A bipartite graph has **two distinct types of nodes** — Users and Movies — and edges only go **between types**, never within. A user never connects to another user directly.

```
Users — rated — Movies

[User 0] —> [Movie 23]
[User 1] —> [Movie 47]
[User 2] —> [Movie 23]
           |
           |> [Movie 91]

943 user nodes      1,682 item nodes
```

### Q4. What is LightGCN and why is it used instead of a full GCN?

**Key insight:** He et al. (2020) showed that removing weight matrices and non-linear activations from GCN actually **improves** recommendation performance. Pure neighbourhood aggregation is sufficient.

| Model     | Key Idea                                     | Best For                      |
|-----------|----------------------------------------------|-------------------------------|
| GCN       | Normalised sum of neighbours + weight matrix | Node classification           |
| GraphSAGE | Samples fixed-size neighbourhood, scalable   | Large graphs, inductive tasks |
| LightGCN  | No weights, no activation, pure aggregation  | Recommendation systems ✓      |

## Q5. What are embeddings and what do they represent?

An embedding is a **dense vector of numbers** (e.g. 64 numbers) that represents an entity. After training:

- Similar users will have similar vectors
- Movies a user would like will have vectors close to that user's vector
- The GNN learns these vectors by training on interaction data

```
user_emb shape : [943, 64]    ← one 64-dim vector per user
item_emb shape : [1682, 64]   ← one 64-dim vector per item

Score(user, movie) = dot_product(user_emb, movie_emb)
High score → strong recommendation
```

## Q6. What is the difference between GCN, GraphSAGE, and LightGCN?

| Aspect                 | GCN                 | GraphSAGE         | LightGCN         |
|------------------------|---------------------|-------------------|------------------|
| Weight matrix          | Yes                 | Yes               | No (removed)     |
| Non-linear activation  | Yes                 | Yes               | No (removed)     |
| Neighbourhood sampling | Full graph          | Fixed-size sample | Full graph       |
| Scalability            | Medium              | High              | Medium           |
| Best use case          | Node classification | Large graphs      | Recommendation ✓ |

## Part 2 — Data & Preprocessing Questions

---

### Q7. Why do we only keep ratings $\geq 4$ (positive interactions)?

GNN-based recommendation systems are trained on **implicit feedback** — the model learns 'this user liked this item.' Ratings below 4 are ambiguous. We treat 4–5 star ratings as **edges in the graph** and ignore the rest.

#### Your actual data (from running the project)

Raw ratings: 100,000  
After filtering ( $\geq 4.0$ ): 55,375 positive interactions  
Users: 943 Items: 1,674  
Graph: 2,617 nodes | 106,982 directed edges (53,491 unique)

### Q8. What is a train/test split in the context of a graph recommendation system?

**Leave-one-out strategy** (standard in recommendation literature):

```
Each user's interactions sorted by timestamp:

[ movie_1, movie_2, ..., movie_N-2 | movie_N-1 | movie_N ]
   Training edges           Validation      Test

Your split result:
Train: 53,491 interactions
Val:   942 interactions (1 per user)
Test:  942 interactions (1 per user)
```

### Q9. What does the `edge_index` tensor look like and why is it shaped `[2, num_edges]`?

PyG represents a graph's edges as a tensor of shape `[2, E]` where `E` = number of edges:

```
edge_index = [[0, 1, 2, ...], ← source nodes
              [5, 6, 7, ...]] ← destination nodes

Edge i goes from: edge_index[0][i] → edge_index[1][i]

Since our graph is undirected, every edge appears twice:
user→item AND item→user
53,491 unique interactions × 2 = 106,982 directed edges
```

## Q10. Why do we shift item IDs by num\_users?

In the graph, all nodes (users AND items) share a single ID space. Without shifting, node 0 could be a user **or** a movie — collision. Solution:

```
Users:  IDs 0 to 942          (943 users)
Items:  IDs 943 to 2616      (items shifted by num_users=943)

movie_id 0    → node 943
movie_id 1    → node 944
movie_id 731  → node 1674

Total nodes: 943 + 1674 = 2617 ← your actual graph had 2,617 nodes
```

## Part 3 — Training Questions

---

### Q11. What is BPR Loss and why not use MSE?

#### The core idea

MSE tries to predict the exact rating number — but we don't care about the number.

We care about the RANKING: is movie A ranked above movie B for this user?

BPR (Bayesian Personalised Ranking) directly optimises for ranking.

```
For each (user, positive_item, negative_item) triple:

pos_score = dot(user_emb, liked_movie_emb)
neg_score = dot(user_emb, random_movie_emb)

BPR Loss = -mean( log( sigmoid( pos_score - neg_score ) ) )
           + L2 regularisation on embeddings

Goal: pos_score >> neg_score

Your training loss went from: 0.6931 → 0.4377 (epoch 29 stopped)
```

### Q12. What is negative sampling and how does it work?

Since users only interact with a tiny fraction of all movies, we don't know which they **dislike** — just which they **ignored**.

- **Positive item:** an item the user actually rated ( $\geq 4$  stars)
- **Negative item:** a randomly sampled item the user never touched
- **Assumption:** unrated items are not preferred (implicit feedback assumption)

### Q13. What is early stopping and why does it matter?

Monitors validation Recall@20 after each epoch. Stops when no improvement for **patience epochs** (default: 20).

#### Your actual early stopping result

Best epoch: 9 (Recall@20 = 0.1360)  
Stopped at: epoch 29 (no improvement for 20 epochs)  
This saved you ~170 epochs of unnecessary training.  
Best model automatically saved to: results/best\_model.pt

### Q14. What is the LightGCN forward pass formula?

```
E0 = embedding.weight           # raw learned embeddings [2617, 64]
E1 = LGConv(E0, edge_index)    # layer 1 aggregation
E2 = LGConv(E1, edge_index)    # layer 2 aggregation
E3 = LGConv(E2, edge_index)    # layer 3 aggregation

E_final = mean(E0, E1, E2, E3) # multi-layer average

user_emb = E_final[:943]         # first 943 rows = users
item_emb = E_final[943:]        # remaining rows = items

LGConv does: xi =  $\sum_j (1 / \sqrt{\deg_i \times \deg_j}) \times x_j$ 
              = degree-normalised sum of neighbour embeddings
```

## Part 4 — Evaluation Questions

---

### Q15. What is Recall@K and how do you calculate it?

For each test user, generate top-K recommendations (excluding seen items). **Recall@K** = fraction of ground-truth items that appeared in the top-K list.

```
Recall@K = |recommended_top_K ∩ relevant_items| / |relevant_items|
```

Example:

```
User liked 5 movies in total
We recommended top-10: 2 of those 5 appear in our list
Recall@10 = 2/5 = 0.40
```

Your actual results:

```
Recall@10 = 0.0849
Recall@20 = 0.1348 ← main reported metric
Recall@50 = 0.2452
```

## Q16. What is NDCG@K?

**NDCG** (Normalised Discounted Cumulative Gain) is like Recall but **position-aware**. A hit at rank 1 scores more than a hit at rank 10.

```
DCG@K =  $\sum \text{rel}_i / \log_2(i + 1)$     for i = 1 to K
IDCG@K = DCG of the perfect ranking (all hits at top)
NDCG@K = DCG@K / IDCG@K

rel_i = 1 if item at rank i is in ground truth, else 0

Your actual results:
NDCG@10 = 0.0431
NDCG@20 = 0.0555    ← main reported metric
NDCG@50 = 0.0773
```

## Q17. Benchmark comparison — where does your model stand?

| Model                | Recall@20 | NDCG@20 | Notes                       |
|----------------------|-----------|---------|-----------------------------|
| Random Baseline      | ~0.010    | ~0.005  | No model, pure random       |
| Matrix Factorisation | ~0.150    | ~0.110  | Classical approach          |
| LightGCN (paper)     | ~0.187    | ~0.133  | He et al. 2020, full tuning |
| Your LightGCN        | 0.1348    | 0.0555  | Early stop at epoch 9 ← you |

### Why your score is slightly below the paper

Early stopping fired at epoch 9 — the model stopped before converging fully.  
This is common when using a small validation sample (500 users for quick eval).  
To get paper-level results: use `--patience 40` and `--epochs 300`.  
The architecture is identical — more training time = higher Recall.

## Part 5 — Anomaly Detection Questions

### Q18. What is GNN-based anomaly detection and how does it work?

After training, every user has a **64-dimensional embedding vector** encoding their taste. Genuine users embed close to their graph neighbours. **Anomalous users** don't fit anywhere.

| Method                 | Logic                                                                                              | Weight |
|------------------------|----------------------------------------------------------------------------------------------------|--------|
| Neighbourhood Distance | L2 distance between user embedding and mean of neighbours. Bots interact randomly → high distance. | 50%    |

| Method           | Logic                                                                                      | Weight |
|------------------|--------------------------------------------------------------------------------------------|--------|
| Isolation Forest | ML anomaly detector on embedding vectors. Flags users in sparse, globally unusual regions. | 30%    |
| Degree + Entropy | Penalises very few interactions OR only popular items (no personal taste).                 | 20%    |

## Q19. What were your actual anomaly detection results?

### Anomaly Detection Results — Real Data

Total users: 943  
 Flagged anomalies: 48 (5.1% flag rate)  
 Score threshold: 0.548 (95th percentile)  
 Score mean: 0.371  
 Score std: 0.130

Top suspicious users:

User 0: score 0.800 — most anomalous in dataset  
 User 255: score 0.701  
 User 838: score 0.697  
 User 629: score 0.692  
 User 416: score 0.691

## Q20. How do you interpret the anomaly score distribution plot?

- The histogram shows a dense cluster of normal users with scores between 0.2–0.5
- The red dashed line at 0.548 is the 95th percentile threshold
- Users to the right of the line are flagged as anomalous
- The box plot clearly shows flagged users have higher scores (median ~0.57) vs normal users (median ~0.35)
- A few extreme outliers (0.70, 0.80) are highly suspicious — likely very sparse or randomly-behaving accounts

## Part 6 — Portfolio & Interview Questions

---

## Q21. Why did you choose GNN over collaborative filtering?

### Say this in an interview

"Collaborative filtering treats the user-item matrix as independent rows and columns. It misses the structural relationships — two users who share many interactions have latent similarity that a GNN can explicitly model through graph propagation.



LightGCN outperforms MF on standard benchmarks precisely because it propagates higher-order connectivity patterns through the bipartite graph."

## Q22. What are the limitations of your project?

Always mention limitations in interviews — it shows engineering maturity.

- **Cold Start Problem:** New users/items with no interactions have no edges. The GNN cannot generate meaningful embeddings for them.
- **Static Graph:** The model is retrained from scratch. It doesn't update in real time as new interactions arrive.
- **No Side Features:** We only use interaction data, not user age, movie genre, director, etc.
- **Scalability:** Full-graph training doesn't scale to millions of nodes without mini-batch sampling (GraphSAGE solves this).
- **Evaluation Protocol:** Leave-one-out gives a noisy estimate with only 1 test item per user. Multiple test items would be more reliable.

## Q23. How would you improve this project?

4. **Add side features:** Movie genre and user demographics as node feature vectors using SAGEConv
5. **Knowledge Graph:** Enrich item representations with KGCN — connecting movies to directors, actors, genres
6. **Scalability:** Replace full-graph training with GraphSAGE neighbourhood sampling for millions of users
7. **Real-time updates:** Streaming graph updates so new interactions are reflected without full retraining
8. **Diversity:** Add a diversity term so recommendations aren't all from the same genre cluster
9. **Explainability:** Use GAT attention weights to show why a movie was recommended

## Q24. Walk me through the end-to-end pipeline.

```
1. DATA LOADING
  MovieLens u.data → filter ratings ≥ 4 → 55,375 interactions
  Leave-one-out split → Train 53,491 | Val 942 | Test 942

2. GRAPH CONSTRUCTION
  Users (0-942) + Items (943-2616) = 2,617 nodes
  Edges = positive interactions (both directions) = 106,982

3. MODEL
  LightGCN: 64-dim embeddings, 3 GCN layers
  Parameters: 167,488 (only the embedding table — no weight matrices)

4. TRAINING
  BPR loss + L2 reg | Adam lr=0.001 | Batch size 1024
```

```
Early stopping patience=20 | Best at epoch 9
```

#### 5. EVALUATION

```
Score all items per user → exclude seen → rank → Recall@20 = 0.1348
```

#### 6. ANOMALY DETECTION

```
Extract embeddings → 3 scoring methods → 48 users flagged
```

## Part 7 — Code Reference

---

### File Overview

| File                 | Purpose                              | Key Functions                                       |
|----------------------|--------------------------------------|-----------------------------------------------------|
| data_loader.py       | Download, clean, split, build graph  | load_data(), build_graph(), sample_negative()       |
| model.py             | LightGCN architecture + BPR loss     | forward(), bpr_loss(), recommend(), save/load()     |
| train.py             | Training loop, early stopping, plots | train(), generate_batch(), evaluate_recall()        |
| evaluate.py          | Full test evaluation + display       | evaluate_full(), recall_at_k(), ndcg_at_k()         |
| main.py              | Single entry point, all modes        | pipeline, train, evaluate, recommend, demo          |
| anomaly_detection.py | 3-method anomaly scoring             | detect_anomalies(), neighbourhood_distance_scores() |

### Key Commands

```
# Full pipeline (train + evaluate):
python main.py

# Train only:
python main.py --mode train --epochs 300 --patience 40

# Evaluate saved model:
python main.py --mode evaluate

# Recommendations for user 42:
python main.py --mode recommend --user_id 42

# Interactive demo:
python main.py --mode demo

# Anomaly detection:
```

```
python src/anomaly_detection.py

# Compare two users in demo:
compare 0 9
```

## Hyperparameter Guide

| Parameter       | Default | What it controls              | Try             |
|-----------------|---------|-------------------------------|-----------------|
| --embedding_dim | 64      | Size of each embedding vector | 32, 64, 128     |
| --num_layers    | 3       | GCN convolution depth         | 2, 3, 4         |
| --lr            | 0.001   | Adam learning rate            | 0.001, 0.0005   |
| --batch_size    | 1024    | Samples per gradient step     | 512, 1024, 2048 |
| --epochs        | 200     | Maximum training epochs       | 200, 300        |
| --patience      | 20      | Early stopping patience       | 20, 40          |
| --reg_lambda    | 1e-4    | L2 regularisation strength    | 1e-4, 1e-5      |
| --top_k         | 20      | K for Recall@K / NDCG@K       | 10, 20, 50      |

## Part 8 — References & Further Study

---

### Papers You Should Know

| Paper                    | Authors           | Key Contribution                                                              |
|--------------------------|-------------------|-------------------------------------------------------------------------------|
| LightGCN (SIGIR 2020)    | He et al.         | Remove weights & activations from GCN for recommendation. This is your model. |
| GCN (ICLR 2017)          | Kipf & Welling    | Original graph convolutional network — foundation of all GNN rec systems.     |
| BPR (UAI 2009)           | Rendle et al.     | Bayesian Personalised Ranking — the loss function you use for training.       |
| GraphSAGE (NeurIPS 2017) | Hamilton et al.   | Scalable GNN via neighbourhood sampling. The next step after LightGCN.        |
| GAT (ICLR 2018)          | Veličković et al. | Graph Attention Networks — adds attention weights, useful for explainability. |

### Learning Resources

- **Stanford CS224W** (Free on YouTube) — best GNN course available. Watch lectures 1–7 minimum.

- **PyTorch Geometric Docs** — [pytorch-geometric.readthedocs.io](https://pytorch-geometric.readthedocs.io) — official API reference and tutorials.
- **Papers With Code** — [paperswithcode.com/task/recommendation-systems](https://paperswithcode.com/task/recommendation-systems) — current SOTA benchmarks.
- **LightGCN Paper** — [arxiv.org/abs/2002.02126](https://arxiv.org/abs/2002.02126) — read the full paper, it is only 9 pages.
- **MovieLens Dataset** — [grouplens.org/datasets/movielens/100k](https://grouplens.org/datasets/movielens/100k) — original dataset you used.

## What to Study Next

10. Read the LightGCN paper — you have already implemented it, now read the original
11. Watch Stanford CS224W Lectures 1–4 to understand graph theory foundations
12. Implement GraphSAGE for scalability — replace LGConv with SAGEConv in model.py
13. Add node features — pass movie genres as initial node feature vectors
14. Try Knowledge Graph enhancement — KGCN or KGAT for richer item representations
15. Deploy as a REST API — wrap main.py recommend mode in a FastAPI endpoint

## Part 9 — Future Upgrades

---

These are the upgrades you should mention when asked **"What would you improve?"** in an interview. They are grouped into four directions. Each one is explained with what it does, why it matters, and what it would realistically improve.

### Category 1 — Architecture Upgrades (better accuracy)

#### 1. Side Features — Node Attributes

##### What it is

Currently the model only uses interaction data — who watched what.

Add movie genre, director, release year and user age/occupation as initial node feature vectors.

Replace LGConv with SAGEConv which supports feature inputs.

**Why it helps:** Two users with identical watching history but different ages may have different future tastes. Node features break this tie. A new movie with no interactions can still get reasonable embeddings from its genre vector.

**How to implement:** Pass a feature matrix X (shape [num\_nodes, feature\_dim]) as the initial node embeddings instead of a random embedding table. Use SAGEConv(in\_channels, out\_channels) layers.

**Expected gain:** +3–5% Recall@20

## 2. Knowledge Graph Enhancement (KGCN / KGAT)

### What it is

Connect items to a knowledge graph — movies link to directors, actors, genres, studios, awards. The model learns richer item representations by traversing KG paths.

Example path: User liked Inception → Christopher Nolan film → sci-fi → Interstellar recommended.

**Why it helps:** Two movies can be similar even if no user watched both — they share the same director. A pure interaction graph misses this entirely. Knowledge graphs bring in this semantic similarity.

**Real-world use:** This is exactly what YouTube and Netflix do in production. Mentioning this in an interview shows you read beyond the basic papers.

**Expected gain:** **+5–8% Recall@20** — the largest single architecture improvement available.

## 3. Graph Attention Networks (GAT)

### What it is

Replace uniform neighbourhood aggregation with learned attention weights.

Each neighbour gets a score — the model decides which interactions matter most.

During aggregation:  $x_i = \sum_j \text{attention}(i,j) \times x_j$  (not plain average).

**Why it helps:** Not all interactions are equally informative. A user who rated 500 movies casually is less informative than one who gave precise 5-star ratings to 20 specific films. Attention learns this automatically.

**Bonus — explainability:** Attention weights tell you WHY a movie was recommended. Output: 'Recommended because you rated Inception 5 stars and users who loved Inception also loved Interstellar.' This is critical for user trust.

**Expected gain:** **+2–4% Recall@20** plus full recommendation explainability.

## 4. Sequential Recommendation (SR-GNN)

### What it is

Model the ORDER in which users watch movies, not just what they watched.

A user who watched action movies this week should get different recs than one who watched romance.

SR-GNN builds a session graph from recent interactions and uses GRU + GNN together.

**Why it helps:** Taste changes over time. LightGCN treats all historical interactions equally — a film watched 5 years ago has the same weight as one watched yesterday. Sequential models capture taste drift.

**Expected gain:** **Captures temporal patterns** — especially important for seasonal content (holiday movies, sports events).

## Category 2 — Scalability Upgrades (handle more users)

### 5. Mini-Batch Sampling — GraphSAGE

#### What it is

Current approach loads the FULL graph into memory — fails beyond ~1M users.

GraphSAGE samples a fixed-size neighbourhood per node (e.g. 10 neighbours per layer).

Training becomes mini-batched: process 512 nodes at a time, not 2,617.

Used in production at Pinterest (3 billion nodes) and LinkedIn.

**Why it matters:** MovieLens has 943 users. Netflix has 250 million. The current architecture cannot scale to production size. GraphSAGE is the standard solution used in every large-scale GNN deployment.

**How to implement:** Replace LGConv with SAGEConv. Use NeighborLoader from PyG instead of loading the full edge\_index. Training loop stays the same, just the data loading changes.

```
# Current (full graph — doesn't scale):
user_emb, item_emb = model(edge_index)    # loads 2,617 nodes at once

# GraphSAGE (mini-batch — scales to millions):
from torch_geometric.loader import NeighborLoader
loader = NeighborLoader(data, num_neighbors=[10, 10, 10],
                       batch_size=512)

for batch in loader:
    out = model(batch.x, batch.edge_index)
```

### 6. Incremental / Online Learning

#### What it is

Current model is retrained from scratch — takes 15–30 minutes every run.

Incremental learning: when user watches a new movie, update only affected embeddings.

New edge (u, i) added → update embeddings of u, i, and their 1-hop neighbours only.

Full retraining:  $O(\text{num\_users} \times \text{epochs})$ . Incremental:  $O(1)$  per new interaction.

**Why it matters:** A recommendation system that only updates nightly misses the most valuable signal — what the user just watched. Real-time updates mean recommendations improve within seconds of each interaction.

**Expected improvement:** **Real-time personalisation** — the single most impactful production upgrade.

## Category 3 — Feature Upgrades (better recommendations)

### 7. Diversity & Serendipity

#### What it is

Current top-10 can be 8 action movies — high accuracy, poor experience.

Add a diversity penalty using Maximal Marginal Relevance (MMR).

For each next recommendation, balance relevance vs. dissimilarity from already-chosen items.

Result: top-10 spans multiple genres while still being personalised.

**Why it matters:** This is called the 'filter bubble' problem. A system that only recommends what it's sure you'll like slowly narrows your world. Diversity keeps users engaged long-term and is a key metric at Netflix and Spotify.

**Formula:**

```
MMR score =  $\lambda$  × relevance(item) - (1- $\lambda$ ) × max_similarity(item, already_recommended)
```

$\lambda = 0.7$  → 70% accuracy, 30% diversity

$\lambda = 1.0$  → pure accuracy (current behaviour)

$\lambda = 0.0$  → pure diversity (random genres)

### 8. Cold Start Handling

#### What it is

New users have zero edges — LightGCN gives them the most popular items (bad personalisation).

New movies have zero edges — LightGCN never recommends them (popularity bias).

Fix: use content features (genre, description embeddings) to initialise new node vectors.

As interactions arrive, refine the embedding using the graph signal.

**Why it matters:** Every new user is a cold start. Every new movie on release day is a cold start. Without this fix, a new film gets zero recommendations for its first week — exactly when it needs promotion most.

**Practical fix for users:** Ask a short onboarding quiz (3–5 genre preferences). Map answers directly to embedding space as a starting point.

### 9. Explainability — 'Why was this recommended?'

#### What it is

Use GNNExplainer or GAT attention weights to trace which interactions drove a recommendation.

Output: 'Recommended because you rated Inception 5 stars, and users who loved Inception also consistently rated Interstellar 5 stars.'

Show the actual graph path that led to the recommendation.

**Why it matters:** Users trust recommendations more when they understand why. Netflix saw a measurable increase in click-through when they added 'Because you watched X' captions. Explainability is also legally required in some regions (EU AI Act).

## Category 4 — Production Upgrades (deploy it)

### 10. REST API Deployment (FastAPI + Docker)

#### What it is

Wrap the recommend() function in a FastAPI endpoint.

Load the model once at startup, serve predictions in <50ms.

GET /recommend?user\_id=42&top\_k=10 → returns JSON with movie list.

Containerise with Docker so it deploys anywhere.

```
# api.py
from fastapi import FastAPI
from src.model import LightGCN
from src.data_loader import load_data

app = FastAPI()
data = load_data()
model = LightGCN.load('results/best_model.pt')

@app.get('/recommend')
def recommend(user_id: int, top_k: int = 10):
    seen = data['train_dict'].get(user_id, set())
    recs = model.recommend(data['graph'].edge_index,
                           user_id, seen, top_k)
    titles = [data['movie_titles'][i] for i in recs]
    return {'user_id': user_id, 'recommendations': titles}

# Run: uvicorn api:app --reload
# Test: curl localhost:8000/recommend?user_id=42
```

### 11. A/B Testing Framework

#### What it is

Compare LightGCN vs Matrix Factorisation vs GraphSAGE on real users.

Split traffic 50/50: half users get Model A, half get Model B.

Measure click-through rate, watch time, and session length — not just Recall@20.

Recall@20 is an offline metric. Real users may prefer the less accurate model.

**Why it matters:** Offline metrics and online metrics often disagree. A model with higher Recall@20 can have lower user satisfaction if its recommendations feel too obvious. A/B testing is how you find out what actually works.



**Key insight for interviews: 'The goal is not to maximise Recall@20 — it is to maximise user engagement. Those are correlated but not identical.'**

## 12. Enhanced Anomaly Detection

### What it is

Current: 3 static scoring methods with a fixed 95th percentile threshold.

Upgrade 1: Dynamic threshold that adapts as the graph grows over time.

Upgrade 2: Temporal patterns — detect sudden behavioural changes (bot activation).

Upgrade 3: Real-time alert system that flags new anomalies as they appear.

Upgrade 4: Feedback loop — when a flagged user is confirmed bot, retrain without them.

**Why it matters:** Your current system scores users on a static snapshot. In production, bot accounts activate suddenly — a user who was normal for 3 months starts making 1,000 interactions per day. Temporal detection catches this; static scoring does not.

## 13. Social Network Extension

### What it is

Add explicit user-user edges (friends, follows) alongside user-item edges.

The graph becomes heterogeneous — two node types, three edge types.

Social connections reinforce recommendations: recommend what your friends liked.

Directly applicable to LinkedIn (jobs), Twitter/X (content), Facebook (events).

**Why it matters:** This extends your project scope from a movie rec system to a general social recommendation framework — a far more valuable portfolio piece. The architecture change is minimal: add user-user edges, add a third edge type to the message passing.

## Summary — All 13 Upgrades at a Glance

| # | Upgrade                            | Category     | Expected Impact                   |
|---|------------------------------------|--------------|-----------------------------------|
| 1 | Side features (node attributes)    | Architecture | +3–5% Recall@20                   |
| 2 | Knowledge graph (KGCN/KGAT)        | Architecture | +5–8% Recall@20                   |
| 3 | Graph attention (GAT)              | Architecture | +2–4% Recall@20 + explainability  |
| 4 | Sequential recommendation (SR-GNN) | Architecture | Temporal taste drift captured     |
| 5 | Mini-batch sampling (GraphSAGE)    | Scalability  | Scales to 100M+ users             |
| 6 | Incremental / online learning      | Scalability  | Real-time personalisation         |
| 7 | Diversity & serendipity (MMR)      | Features     | Prevents filter bubbles           |
| 8 | Cold start handling                | Features     | New users/items usable from day 1 |

| #  | Upgrade                            | Category   | Expected Impact               |
|----|------------------------------------|------------|-------------------------------|
| 9  | Explainability (GNExplainer / GAT) | Features   | User trust + legal compliance |
| 10 | REST API deployment (FastAPI)      | Production | Real users, real traffic      |
| 11 | A/B testing framework              | Production | Data-driven model selection   |
| 12 | Enhanced anomaly detection         | Production | Live fraud detection          |
| 13 | Social network extension           | Production | Broader applicability         |

### 30-second interview answer

"There are four directions I would take this project. First, architecture — add side features like movie genre and user demographics as node feature vectors, which gives +3–5% Recall@20, and a knowledge graph to connect movies to directors and actors for +5–8% more. Second, scalability — replace full-graph training with GraphSAGE mini-batch sampling so it handles millions of users like Pinterest does in production. Third, features — add explainability using attention weights and fix the cold start problem. Fourth, production — wrap it in a FastAPI REST endpoint and add A/B testing to compare models on real users, because offline Recall@20 and real user satisfaction are correlated but not identical."